Programación Concurrente y Paralela

Práctica 7: Lógica para modelado de concurrencia

2do Cuatrimestre 2026

Ejercicio 1. Considere el programa `incr()`, que incrementa una variable `x`:

```
int x = 0

void incr() {
p1:    int loc = x
p2:    x = loc + 1
}
```

Cada línea etiquetada se ejecuta de forma atómica. Escriba el diagrama LTS para `incr()||incr()`.

Ejercicio 2. Considere el siguiente programa:

```
Lock a, b
void p() {                    void q() {
p1: a.lock()                  q1: b.lock()
p2: b.lock()                  q2: a.lock()
p3: usar()                    q3: usar()
p4: b.unlock()                q4: a.unlock()
p5: a.unlock()                q5: b.unlock()
}                             }
```

Construya el LTS de `p()||q()` y encuentre el camino más corto a un deadlock. Proponga dos modificaciones que lo eliminen. En una, un thread se rinde y libera lo que tiene (considere la instrucción atómica `tryLock()`: para `x.tryLock()`, si `x` está libre lo toma y devuelve **`true`**, y si no devuelve **`false`** sin bloquearse.). En la otra, se cambia el orden de adquisición. Muestre que ninguno de los dos LTS resultantes tiene deadlock.

Ejercicio 3. Considere el siguiente programa en Go:

```
c1 := make(chan int)
c2 := make(chan int)

go func() { c2 <- 2; <-c1 }()
c1 <- 1
<-c2
```

Modele el programa como composición de LTS, primero con los canales tal como están y después creándolos con `make(chan int, 1)`. Determine si cada versión puede quedar bloqueada.

Ejercicio 4. Dadas las trazas

- $\sigma_1 = \{p\}\ \{p,q\}\ \{q\}\ (\{p\})^\omega$

- $\sigma_2 = (\{p\}\ \{q\})^\omega$
- $\sigma_3 = \{q\}\ \emptyset\ (\{p, q\})^\omega$
- $\sigma_4 = \{p\}\ \emptyset\ (\{p, q\}\ \{p\}\ \emptyset)^\omega$

decida cuáles satisfacen $\Box\Diamond q$, $\Diamond\Box p$, $p\ \mathbf{U}\ q$, $\Box(p \to \Diamond q)$, $\bigcirc\bigcirc q$ y $\Box(p\ \mathbf{U}\ q)$.

Ejercicio 5. Decida cuáles de las siguientes equivalencias son correctas. Demuéstrelas o dé un contraejemplo.

a) $\Diamond(\varphi \wedge \psi) \equiv \Diamond\varphi \wedge \Diamond\psi$

b) $\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi$

c) $\bigcirc\Diamond\varphi \equiv \Diamond\bigcirc\varphi$

d) $\Box\Diamond\varphi \to \Box\Diamond\psi \equiv \Box(\varphi \to \Diamond\psi)$

e) $\Box\varphi \to \Diamond\psi \equiv \varphi\ \mathbf{U}\ (\psi \vee \neg\varphi)$

Sugerencia para el ítem e: reescriba el lado izquierdo como una fórmula de la forma $\Diamond\chi$, y después piense en la *primera* posición en la que vale χ.

Ejercicio 6. Una fórmula es *satisfacible* si existe al menos una traza que la satisface, y es *válida* si todas las trazas la satisfacen. Para cada una de las siguientes fórmulas, decida si es satisfacible y si es válida.

a) $\bigcirc\bigcirc a \to \bigcirc a$

b) $\bigcirc(a \vee \Diamond a) \to \Diamond a$

c) $\Box a \to \neg\bigcirc(\neg a \wedge \Box\neg a)$

d) $(\Box a)\ \mathbf{U}\ (\Diamond b) \to \Box(a\ \mathbf{U}\ \Diamond b)$

e) $\Diamond b \to (a\ \mathbf{U}\ b)$

Ejercicio 7. Dos threads acceden a una sección crítica protegida por un lock. Las proposiciones son $request_i$, que vale desde que el thread i llama a `lock()` hasta que libera el lock, $critical_i$, que vale mientras el thread i está en la sección crítica, y $release_i$, que vale en el instante en que el thread i llama a `unlock()`. Escriba en LTL las siguientes propiedades y clasifique cada una como safety o liveness:

a) Exclusión mutua: un solo thread está en la sección crítica a la vez.

b) Permanencia finita: ningún thread se queda en la sección crítica para siempre.

c) Ausencia de inanición: si un thread pide el lock, eventualmente entra a la sección crítica.

d) Acceso alternado: los threads se turnan estrictamente para entrar a la sección crítica.

e) Si el thread 1 pide el lock infinitas veces, entra a la sección crítica infinitas veces.

f) Si a partir de algún momento el thread 1 pide el lock sin parar, entra a la sección crítica infinitas veces.

g) Mientras el thread 1 espera, el thread 2 entra a la sección crítica a lo sumo una vez.

Ejercicio 8. Defina en LTL los siguientes operadores:

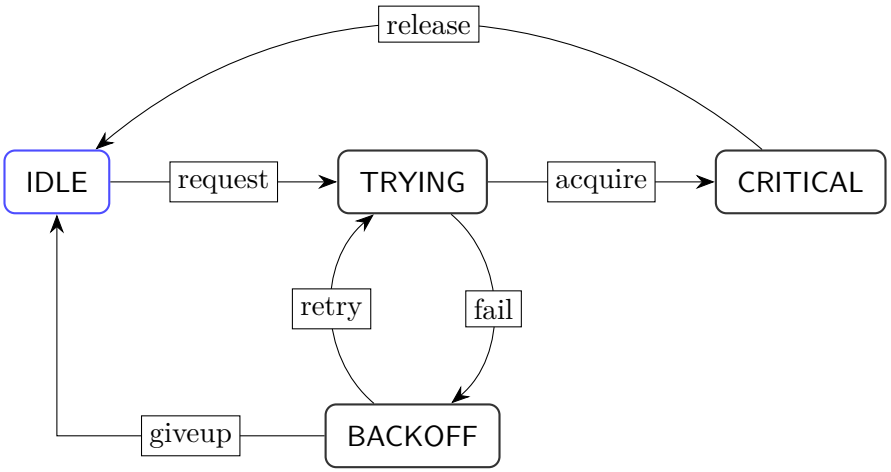
- $\varphi \,\mathbf{N}\, \psi$ ("en el próximo"): la próxima vez que vale ψ, también vale φ.
- $\varphi \,\mathbf{M}\, \psi$ ("mientras"): φ vale al menos mientras valga ψ.
- $\varphi \,\mathbf{B}\, \psi$ ("antes"): si ψ vale alguna vez, φ vale antes.

Ejercicio 9. Un thread usa un lock en un loop: lo toma, lo suelta, lo vuelve a tomar, y así para siempre. Con la proposición *held*, que vale mientras el thread tiene el lock, la propiedad "el thread alterna entre tener el lock y no tenerlo" se escribió como

$$\Box\Big((\neg \mathit{held} \to \bigcirc \mathit{held}) \wedge (\mathit{held} \to \bigcirc \neg \mathit{held})\Big)$$

La fórmula vale para el thread solo, pero deja de valer si se lo compone con otro thread que nunca usa el lock, por ejemplo uno que solo incrementa un contador propio en un loop. Reescriba la propiedad para que no se vea afectada por esa composición.

Ejercicio 10. El siguiente LTS abstrae a un thread que usa `tryLock()` con backoff. `tryLock()` intenta tomar el lock sin bloquearse: si está libre lo toma, y si no, devuelve **`false`** enseguida. Con *backoff*, cuando el intento falla el thread espera un rato antes de volver a intentar, para no competir todo el tiempo por el lock. Después de esperar puede reintentar (`retry`) o rendirse y volver al principio (`giveup`). En el LTS, BACKOFF es ese estado de espera. La competencia con los demás threads aparece como la elección entre `acquire` y `fail`.



En las fórmulas, las proposiciones son los nombres de los estados: en cada posición de la traza vale el nombre del estado en el que está el thread. Decida si valen las siguientes fórmulas:

- $\Box(\text{TRYING} \to \Diamond\text{CRITICAL})$
- $\Box(\text{TRYING} \to \Diamond(\text{CRITICAL} \vee \text{IDLE}))$
- $\Box(\text{BACKOFF} \to \Box\neg\text{CRITICAL})$
- $\Box(\text{BACKOFF} \to \Diamond\text{TRYING})$

Para las que no valen, dé el contraejemplo. Identifique los conjuntos terminales y diga cuáles fórmulas pasan a valer bajo fair choice.

Ejercicio 11. Dibuje un LTS observador con estado de error para la siguiente propiedad: nadie usa el recurso sin haberlo adquirido, y no hay dos `acquire` sin un `release` en el medio. El alfabeto es {`acquire`, `use`, `release`}. Escriba la misma propiedad en LTL, tomando como proposiciones las acciones: en cada posición de la traza vale solo la acción de ese paso.

Ejercicio 12. Dos threads ($i = 0, 1$) ejecutan:

```
int s = 1
bool y[2] = {false, false}

loop {
  noncritical()
  (y[i], s) = (true, i)
  await (!y[1-i] || s == i)
  critical()
  y[i] = false
}
```

donde `(y[i], s) = (true, i)` es una asignación múltiple atómica.

a) Construya el LTS del programa.

b) Escriba en LTL exclusión mutua y ausencia de inanición, usando proposiciones sobre el estado, como $critical_i$, que vale cuando el thread i está en `critical()`.

c) Verifique ambas propiedades sobre el LTS. Para las que fallen, dé un contraejemplo y decida si pasan a valer bajo fair choice.

Ejercicio 13. Dos threads ejecutan:

```
int y1 = 0, y2 = 0

thread 1:
loop {
  noncritical()
  y1 = y2 + 1
  await (y2 == 0 || y1 < y2)
  critical()
  y1 = 0
}
```

```
thread 2:
loop {
  noncritical()
  y2 = y1 + 1
  await (y1 == 0 || y2 < y1)
  critical()
  y2 = 0
}
```

Las asignaciones `y1 = y2 + 1` y `y2 = y1 + 1` son atómicas.

a) Construya el LTS alcanzable restringido a `y1` ≤ 2 y `y2` ≤ 2.

b) Muestre una ejecución que pruebe que el LTS completo es infinito.

c) Escriba en LTL exclusión mutua, ausencia de deadlock y ausencia de inanición, usando proposiciones sobre el estado, como $critical_i$, que vale cuando el thread i está en `critical()`. Verifique las tres propiedades.

Ejercicio 14. Un servidor atiende a dos clientes. Los pedidos llegan por un canal, y las respuestas vuelven por otro canal que comparten los dos clientes:

```
requests := make(chan int)
responses := make(chan int)

// server
for { i := <-requests; responses <- i }

// client i (i = 1, 2)
for { requests <- i; r := <-responses; use(r) }
```

a) Modele el sistema como composición de LTS, con los canales tal como están y con `responses` creado con `make(chan int, 1)`.

b) Escriba en LTL, usando proposiciones sobre el estado, que cada cliente recibe siempre la respuesta a su propio pedido, y que todo cliente que manda un pedido termina recibiendo una respuesta.

c) Verifique las dos propiedades en las dos versiones. Para las que fallen, dé un contraejemplo y decida si pasan a valer bajo fair choice.